

Ridge and Lasso

S. Tonini, F. Chiaromonte (special thanks to J. Di Iorio, L. Insolia and L. Testa)

March 5th 2026

Contents

Introduction	1
Libraries	1
Data	1
Penalized regression	2
Ridge	2
Lasso	7
Unbiasing the LASSO	11
A more realistic setting	11
Digging in the irrepresentability condition	18
Nonconvex penalization methods	20
Ridge and LASSO with Information Criterion	21

Introduction

Libraries

We are going to use a few libraries:

- glmnet: Lasso and Elastic-Net Regularized Generalized Linear Models
- tidyverse:
- caret: Classification and Regression Training
- ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics
- corrplot: Correlation matrix plotting

```
library(glmnet)      # ridge and lasso for GLMs
library(tidyverse)    # data manipulation and visualization
library(caret)        # statistical learning techniques
library(ggplot2)      # plots
library(corrplot)     # correlation matrix plotting
```

Data

We will use the **Body Fat dataset** (which is available in the [Datasets folder](#) of our course).

The data concerns a sample of 252 men, and contains 15 variables:

- Density of the body, determined from underwater weighing
- Percentage of body fat, calculated as a function of the Density according to Siri's equation: $(495/\text{Density}) - 450$.
- Indicator for Age group (binary; 0: up to 45 years, 1: over 45)
- Weight (lbs)
- Height (inches)
- Neck circumference (cm)
- Chest circumference (cm)
- Abdomen circumference (cm)
- Hip circumference (cm)
- Thigh circumference (cm)
- Knee circumference (cm)
- Ankle circumference (cm)
- Biceps circumference (cm)
- Forearm circumference (cm)
- Wrist circumference (cm)

We want to understand whether we can reliably describe and predict body fat percentage on the basis of these variables, using regression methods. For age, we only have a binary indicator separating men below and above 45 years. The body measurements, on the other hand, are all continuous variables. Please see the [data description file](#) for more details.

```
df <- read.table('BODY_FAT.TXT', header=TRUE)
names(df)

## [1] "Density" "SiriBF." "Over45" "Weight" "Height" "NeckC"
## [7] "ChestC" "AbdomenC" "HipC" "ThighC" "KneeC" "AnkleC"
## [13] "BicepsC" "ForearmC" "WristC"
```

We want to predict “SiriBF.” using the other features, aside from “Density”. So we drop the “Density” column.

```
df <- df[, -1]
```

Penalized regression

We will perform ridge/lasso penalization through the **glmnet** package. Let us identify predictors and response variable

```
# getting the predictors
x_var <- data.matrix(df[, -1]) # NOTE: glmnet requires a matrix structure
# getting the response variable
y_var <- df[, "SiriBF."]
```

Let's have a look at the glmnet function:

```
help(glmnet)
```

Note that:

- input matrix
- response variable
- α is the elastic-net mixing parameter with range $[0, 1]$. Namely, $\alpha = 1$ is the lasso (default) and $\alpha = 0$ is the ridge.
- standardize is a logical flag for x variable standardization, prior to fitting the model sequence. The coefficients are always returned on the original scale. Default is standardize=TRUE.

Ridge

To perform ridge regression, we run `glmnet` with $\alpha = 0$. The λ 's sequence is internally computed by the package itself – although a user-defined sequence can be provided as a *lambda* argument.

```
ridge <- glmnet(x_var, y_var, alpha=0)
summary(ridge)
```

```
##           Length Class      Mode
## a0          100  -none-   numeric
## beta        1300 dgCMatrx S4
## df           100  -none-   numeric
## dim           2  -none-   numeric
## lambda       100  -none-   numeric
## dev.ratio    100  -none-   numeric
## nulldev        1  -none-   numeric
## npasses        1  -none-   numeric
## jerr           1  -none-   numeric
## offset         1  -none-   logical
## call           4  -none-    call
## nobs           1  -none-   numeric
```

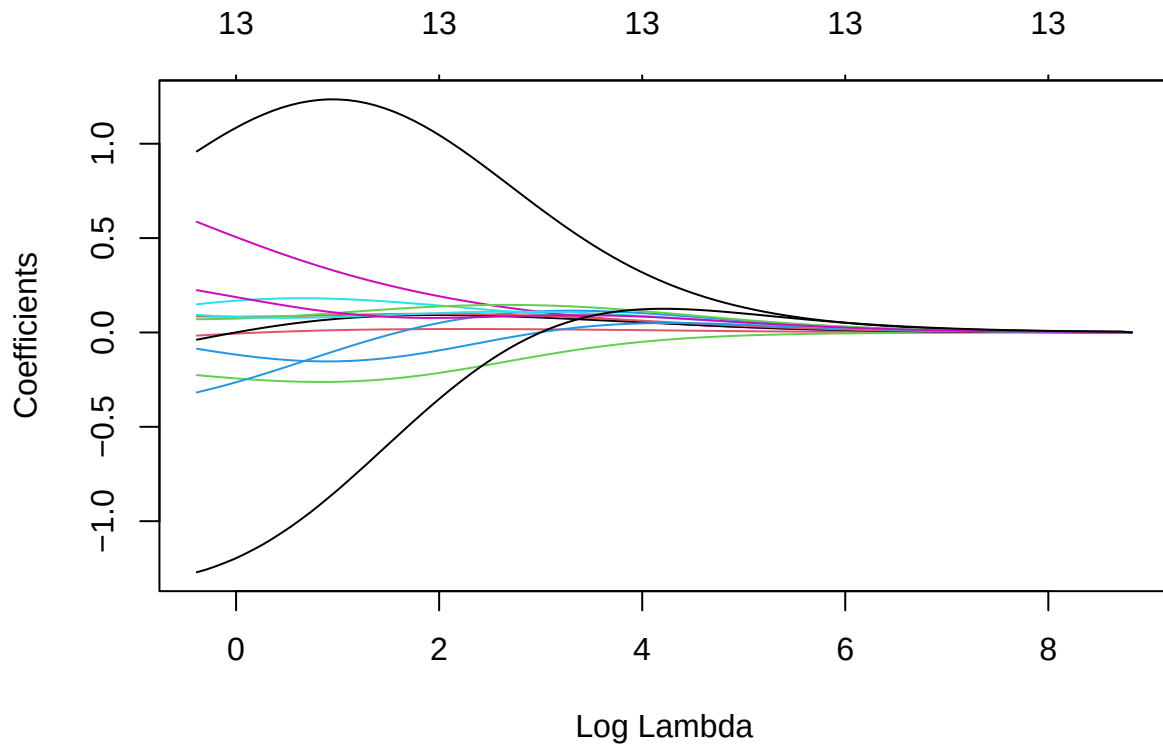
The summary is quite different than the one for linear regression, since ridge regression requires the tuning of λ . The code above fits a ridge regression for each λ value, and we have access to each of these model estimates.

We can plot the regularization path as follows:

```
dim(ridge$beta)
```

```
## [1] 13 100
```

```
plot(ridge, xvar="lambda")
```



We can automate the task of finding the optimal lambda value using the `cv.glmnet` function. This performs a k-fold cross-validation for glmnet, produces a plot, and returns “optimal” λ values.

```
cv_ridge <- cv.glmnet(x_var, y_var, alpha = 0)
cv_ridge
```

```
##
## Call:  cv.glmnet(x = x_var, y = y_var, alpha = 0)
##
## Measure: Mean-Squared Error
##
##      Lambda Index Measure      SE Nonzero
## min 0.6794   100   22.87 1.409         13
## 1se 1.0818    95   24.21 1.414         13
```

Two particular values of λ are highlighted: the minimum (min) and the largest value of lambda such that error is within 1 standard error of the minimum (1se).

```
cv_ridge$lambda.min
```

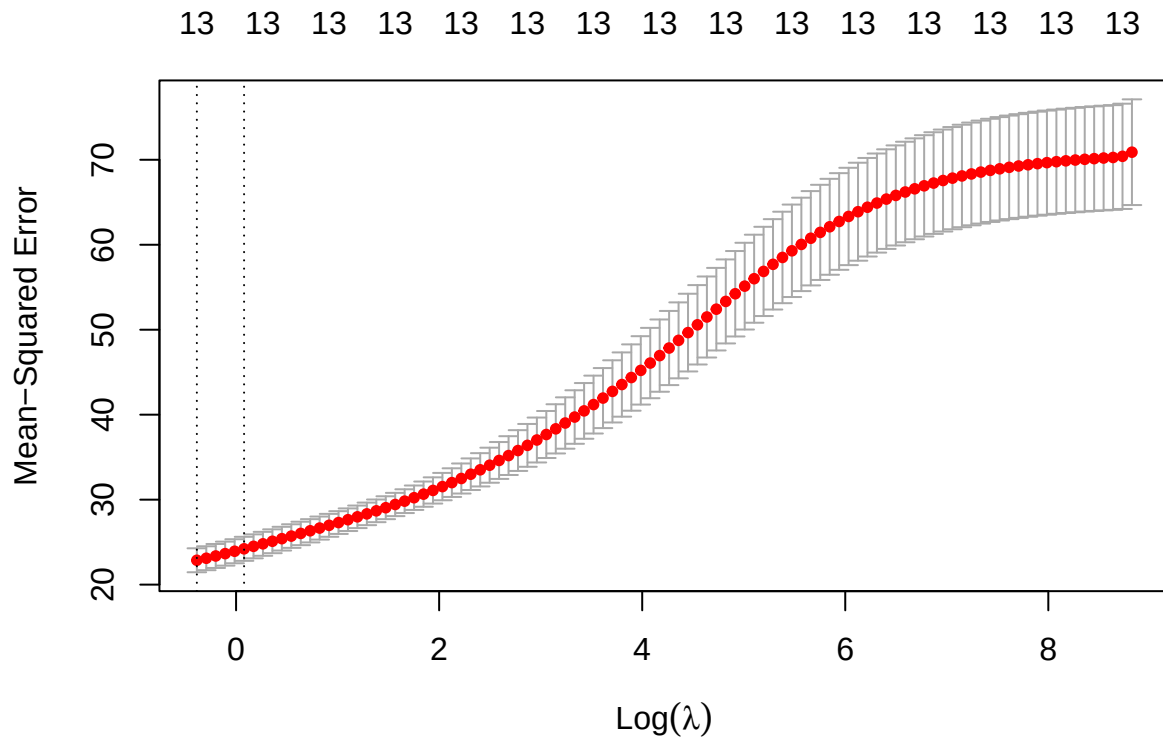
```
## [1] 0.6793883
```

```
cv_ridge$lambda.1se
```

```
## [1] 1.081778
```

We can visualize them in this way:

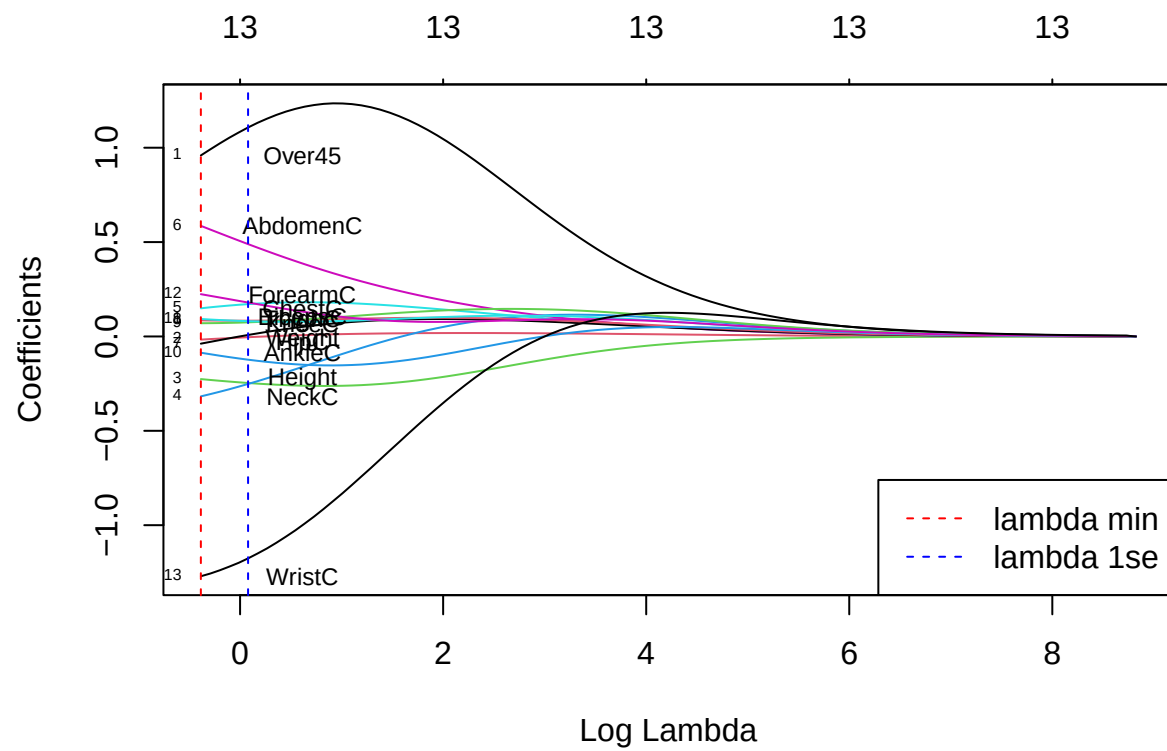
```
plot(cv_ridge)
```



Let us see again how the regression coefficients change by modifying λ , highlighting the min and 1se values:

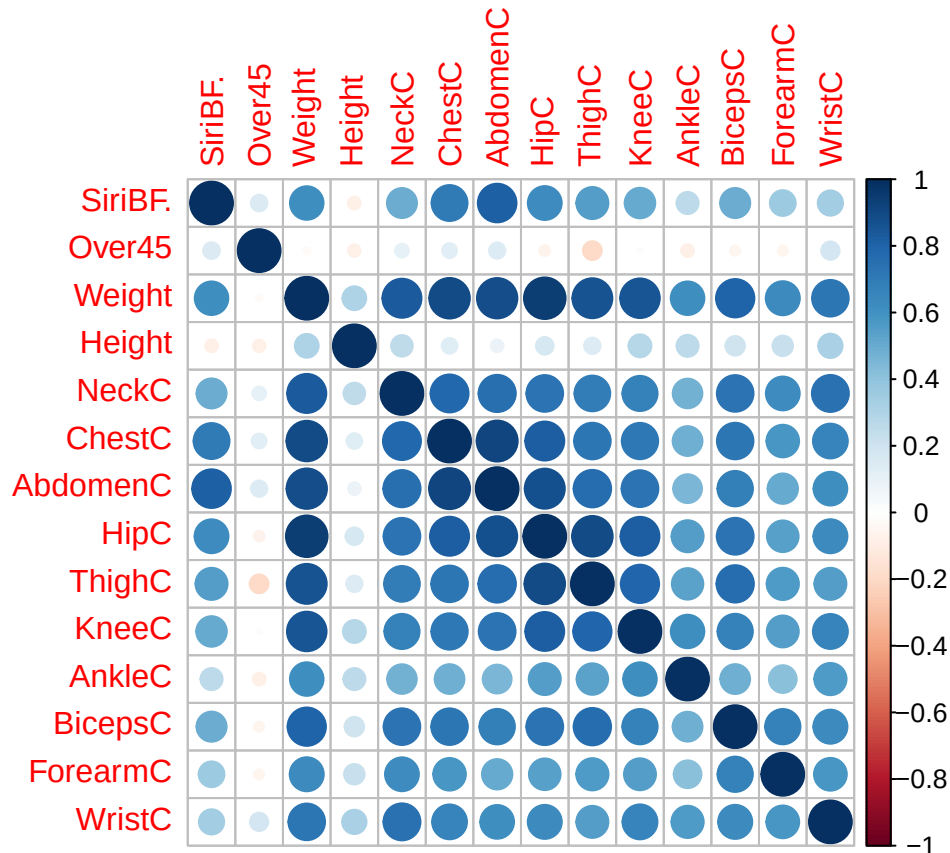
```
lbs_fun <- function(fit, offset_x=1, ...) {
  L <- length(fit$lambda)
  x <- log(fit$lambda[L]) + offset_x
  y <- fit$beta[, L]
  labs <- names(y)
  text(x, y, labels=labs, cex=0.75, ...)
}

plot(ridge, xvar = "lambda", label=T)
lbs_fun(ridge)
abline(v=log(cv_ride$lambda.min), col = "red", lty=2)
abline(v=log(cv_ride$lambda.1se), col="blue", lty=2)
legend(x = "bottomright",
  legend = c("lambda min", "lambda 1se"),
  lty = c(2, 2),
  col = c("red", "blue"))
```



WristC has a very strong negative effect on the response. Why is that? We can find a partial answer by looking at the correlation matrix!

```
corrplot(cor(df))
```



Let's re-fit the model and see the estimates associated to the minimum λ .

```
min_ridge <- glmnet(x_var, y_var, alpha=0, lambda= cv_ridge$lambda.min)
coef(min_ridge)
```

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) -7.71880406
## Over45      0.95953292
## Weight     -0.01619205
## Height     -0.22630209
## NeckC      -0.31617445
## ChestC      0.14889642
## AbdomenC    0.58565111
## HipC       -0.03875703
## ThighC      0.08495378
## KneeC      0.06923480
## AnkleC     -0.08687724
## BicepsC     0.09165336
## ForearmC    0.22347815
## WristC     -1.27272327
```

We can use this model to make predictions on the training set.

```
# Make predictions on the training data
predictions <- min_ridge %>% predict(x_var) %>% as.vector()
# Model performance metrics
data.frame(
```

```

RMSE = RMSE(predictions, y_var),
Rsquare = R2(predictions, y_var)
)

```

```

##          RMSE  Rsquare
## 1 4.494532 0.714938

```

Be careful though! We are making predictions and assessing the goodness of fit based on training data. Is it the best choice? Do you have any other suggestions?

Lasso

Let us now perform Lasso regression using the **glmnet** package. We follow the same approach as in Ridge regression, but set $\alpha = 1$.

```

lasso <- glmnet(x_var, y_var, alpha=1)
summary(lasso)

```

```

##          Length Class      Mode
## a0           77  -none-  numeric
## beta        1001 dgCMatrix S4
## df           77  -none-  numeric
## dim           2  -none-  numeric
## lambda       77  -none-  numeric
## dev.ratio    77  -none-  numeric
## nulldev       1  -none-  numeric
## npasses       1  -none-  numeric
## jerr          1  -none-  numeric
## offset        1  -none-  logical
## call          4  -none-   call
## nobs          1  -none-  numeric

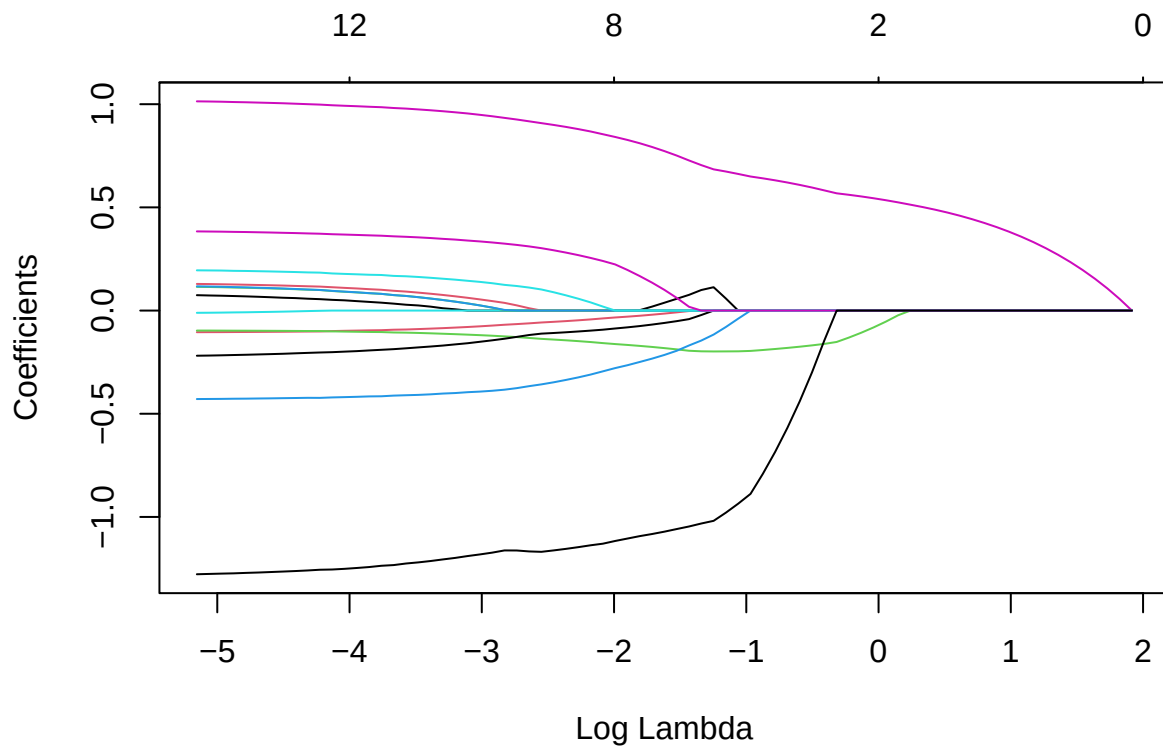
```

Let's have a look at the selection path:

```

plot(lasso, xvar="lambda")

```

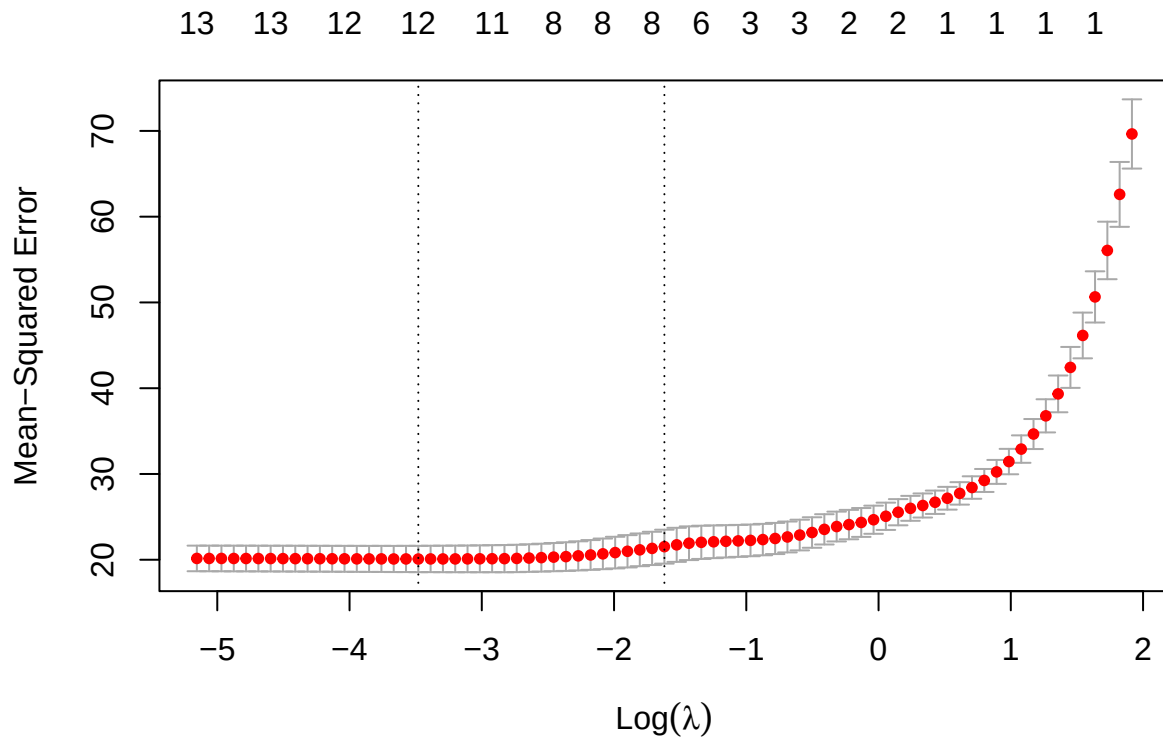



Once again, we need to tune the sparsity parameter λ . We use k-fold cross-validation through the `cv.glmnet` function.

```
cv_lasso <- cv.glmnet(x_var, y_var, alpha = 1)
cv_lasso

##
## Call:  cv.glmnet(x = x_var, y = y_var, alpha = 1)
##
## Measure: Mean-Squared Error
##
##      Lambda Index Measure      SE Nonzero
## min 0.03081   59  20.09 1.535        12
## 1se 0.19805   39  21.53 1.976         8

plot(cv_lasso)
```

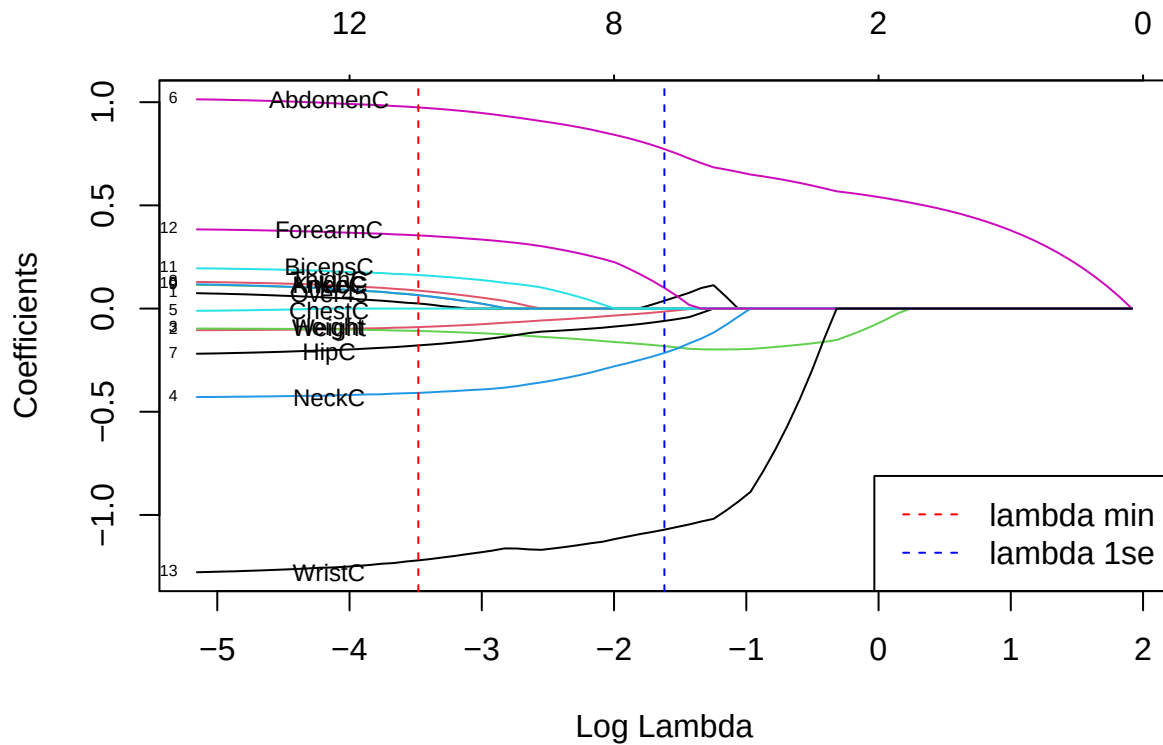


Also here, it outputs the min and the 1se λ . As expected, the number of non-zero coefficients (which is printed on top of the previous plot) is lower than the one for Ridge regression.

Let us see again how the regression coefficients change by modifying λ :

```
lbs_fun <- function(fit, offset_x=1, ...) {
  L <- length(fit$lambda)
  x <- log(fit$lambda[L]) + offset_x
  y <- fit$beta[, L]
  labs <- names(y)
  text(x, y, labels=labs, cex=0.75, ...)
}

plot(lasso, xvar = "lambda", label=T)
lbs_fun(lasso)
abline(v=log(cv_lasso$lambda.min), col = "red", lty=2)
abline(v=log(cv_lasso$lambda.1se), col="blue", lty=2)
legend(x = "bottomright",
       legend = c("lambda min", "lambda 1se"),
       lty = c(2, 2),
       col = c("red", "blue"))
```



Let us rebuild the model and compare the estimated coefficients for min and 1se λ .

```
min_lasso <- glmnet(x_var, y_var, alpha=1, lambda= cv_lasso$lambda.min)
se_lasso <- glmnet(x_var, y_var, alpha=1, lambda= cv_lasso$lambda.1se)

lasso_mat <- cbind(coef(min_lasso), coef(se_lasso))
colnames(lasso_mat) <- c("min", "1se")
lasso_mat
```

```
## 14 x 2 sparse Matrix of class "dgCMatrix"
##           min      1se
## (Intercept) -16.59587742 -6.01107172
## Over45      0.01905466  0.03093253
## Weight     -0.09058505 -0.01415629
## Height     -0.10783072 -0.18224635
## NeckC      -0.40725292 -0.21186539
## ChestC      .          .
## AbdomenC    0.97590540  0.77362334
## HipC       -0.17880465 -0.06291140
## ThighC      0.08848470  .
## KneeC       0.06525229  .
## AnkleC      0.06509234  .
## BicepsC     0.16233628  .
## ForearmC    0.35412766  0.09800305
## WristC     -1.21494502 -1.07588345
```

We can use this model to make predictions on the training set.

```

# Make predictions on the training data
predictions <- se_lasso %>% predict(x_var) %>% as.vector()
# Model performance metrics
data.frame(
  RMSE = RMSE(predictions, y_var),
  Rsquare = R2(predictions, y_var)
)

##      RMSE      Rsquare
## 1 4.391121 0.7252105

```

Unbiasing the LASSO

You have learnt that LASSO estimates are biased downwards, i.e. the coefficients are shrunk towards 0. An approach to debias them is the following:

- run a LASSO on the original data set;
- run a OLS on the set of variables selected by the LASSO in the previous step.

Perhaps, compare the following two regression methods:

```

predictions_bias <- min_lasso %>% predict(x_var) %>% as.vector()

coefs = which(coef(min_lasso)!=0) - 1
coefs = coefs[2:length(coefs)]
prediction_2step <- lm(y_var ~ ., data = as.data.frame(x_var[, coefs])) %>% predict(as.data.frame(x_var[, coefs]))

# Model performance metrics
data.frame(
  RMSE_bias = RMSE(predictions_bias, y_var),
  Rsquare_bias = R2(predictions_bias, y_var),
  RMSE_2step = RMSE(prediction_2step, y_var),
  Rsquare_2step = R2(prediction_2step, y_var)
)

##      RMSE_bias Rsquare_bias RMSE_2step Rsquare_2step
## 1  4.224679    0.7442438   4.216389    0.7451481

```

A more realistic setting

Let's try to overfit and look at the MSE increase again!

```

# take all pairwise interactions
dim(x_var)

## [1] 252 13

newx <- as.data.frame(x_var)
newx <- model.matrix(~ .^2, data=newx)
dim(newx)

## [1] 252 92

colnames(newx)

## [1] "(Intercept)"      "Over45"           "Weight"
## [4] "Height"           "NeckC"            "ChestC"
## [7] "AbdomenC"         "HipC"             "ThighC"
## [10] "KneeC"            "AnkleC"           "BicepsC"

```

```
## [13] "ForearmC"      "WristC"        "Over45:Weight"
## [16] "Over45:Height" "Over45:NeckC"  "Over45:ChestC"
## [19] "Over45:AbdomenC" "Over45:HipC"   "Over45:ThighC"
## [22] "Over45:KneeC"    "Over45:AnkleC" "Over45:BicepsC"
## [25] "Over45:ForearmC" "Over45:WristC" "Weight:Height"
## [28] "Weight:NeckC"    "Weight:ChestC" "Weight:AbdomenC"
## [31] "Weight:HipC"     "Weight:ThighC" "Weight:KneeC"
## [34] "Weight:AnkleC"   "Weight:BicepsC" "Weight:ForearmC"
## [37] "Weight:WristC"   "Height:NeckC"   "Height:ChestC"
## [40] "Height:AbdomenC" "Height:HipC"    "Height:ThighC"
## [43] "Height:KneeC"    "Height:AnkleC"  "Height:BicepsC"
## [46] "Height:ForearmC" "Height:WristC"  "NeckC:ChestC"
## [49] "NeckC:AbdomenC"  "NeckC:HipC"     "NeckC:ThighC"
## [52] "NeckC:KneeC"     "NeckC:AnkleC"   "NeckC:BicepsC"
## [55] "NeckC:ForearmC"  "NeckC:WristC"   "ChestC:AbdomenC"
## [58] "ChestC:HipC"     "ChestC:ThighC"  "ChestC:KneeC"
## [61] "ChestC:AnkleC"   "ChestC:BicepsC" "ChestC:ForearmC"
## [64] "ChestC:WristC"   "AbdomenC:HipC"  "AbdomenC:ThighC"
## [67] "AbdomenC:KneeC"  "AbdomenC:AnkleC" "AbdomenC:BicepsC"
## [70] "AbdomenC:ForearmC" "AbdomenC:WristC" "HipC:ThighC"
## [73] "HipC:KneeC"      "HipC:AnkleC"    "HipC:BicepsC"
## [76] "HipC:ForearmC"   "HipC:WristC"    "ThighC:KneeC"
## [79] "ThighC:AnkleC"   "ThighC:BicepsC" "ThighC:ForearmC"
## [82] "ThighC:WristC"   "KneeC:AnkleC"   "KneeC:BicepsC"
## [85] "KneeC:ForearmC"  "KneeC:WristC"   "AnkleC:BicepsC"
## [88] "AnkleC:ForearmC" "AnkleC:WristC"  "BicepsC:ForearmC"
## [91] "BicepsC:WristC"  "ForearmC:WristC"
```

```
# take their squared terms too!
```

```
newx <- as.matrix(newx)
newx <- cbind(newx, newx[, 2:ncol(newx)]^2)
dim(newx)
```

```
## [1] 252 183
```

```
cv_lasso <- cv.glmnet(newx, y_var, alpha = 1)
cv_lasso
```

```
##
```

```
## Call: cv.glmnet(x = newx, y = y_var, alpha = 1)
```

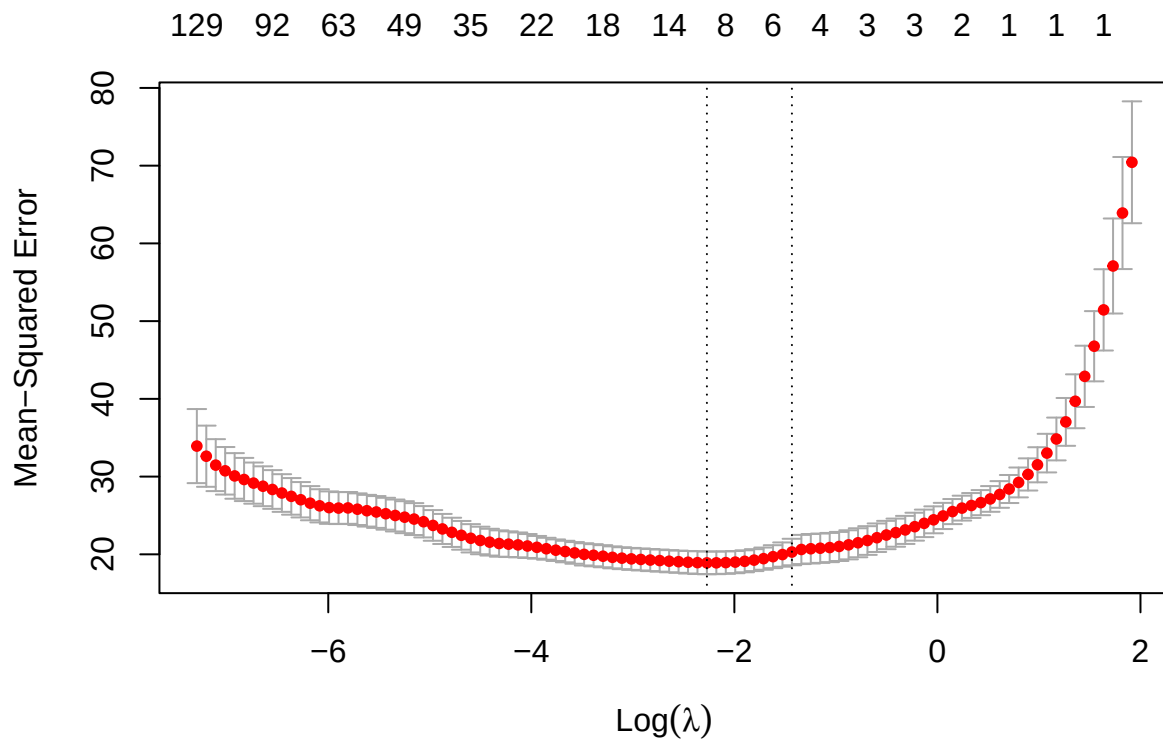
```
##
```

```
## Measure: Mean-Squared Error
```

```
##
```

```
##      Lambda Index Measure      SE Nonzero
## min 0.1033    46    18.9 1.464         8
## 1se 0.2386    37    20.3 1.706         6
```

```
plot(cv_lasso)
```



```
coef(cv_lasso)
```

```
## 184 x 1 sparse Matrix of class "dgCMatrix"
##                               s1
## (Intercept)                -4.228597e+01
## (Intercept)                  .
## Over45                      .
## Weight                      .
## Height                      .
## NeckC                       .
## ChestC                      .
## AbdomenC                    7.877924e-01
## HipC                        .
## ThighC                      .
## KneeC                       .
## AnkleC                      .
## BicepsC                     .
## ForearmC                    .
## WristC                      .
## Over45:Weight               .
## Over45:Height               .
## Over45:NeckC                .
## Over45:ChestC               .
## Over45:AbdomenC             .
## Over45:HipC                 .
## Over45:ThighC               .
```

```

## Over45:KneeC .
## Over45:AnkleC .
## Over45:BicepsC .
## Over45:ForearmC .
## Over45:WristC .
## Weight:Height .
## Weight:NeckC .
## Weight:ChestC .
## Weight:AbdomenC .
## Weight:HipC .
## Weight:ThighC .
## Weight:KneeC .
## Weight:AnkleC .
## Weight:BicepsC .
## Weight:ForearmC .
## Weight:WristC .
## Height:NeckC .
## Height:ChestC .
## Height:AbdomenC .
## Height:HipC .
## Height:ThighC .
## Height:KneeC .
## Height:AnkleC .
## Height:BicepsC .
## Height:ForearmC .
## Height:WristC .
## NeckC:ChestC .
## NeckC:AbdomenC .
## NeckC:HipC .
## NeckC:ThighC .
## NeckC:KneeC .
## NeckC:AnkleC .
## NeckC:BicepsC .
## NeckC:ForearmC .
## NeckC:WristC .
## ChestC:AbdomenC .
## ChestC:HipC .
## ChestC:ThighC .
## ChestC:KneeC .
## ChestC:AnkleC .
## ChestC:BicepsC .
## ChestC:ForearmC .
## ChestC:WristC .
## AbdomenC:HipC .
## AbdomenC:ThighC .
## AbdomenC:KneeC .
## AbdomenC:AnkleC .
## AbdomenC:BicepsC .
## AbdomenC:ForearmC 1.262034e-04
## AbdomenC:WristC .
## HipC:ThighC .
## HipC:KneeC .
## HipC:AnkleC .
## HipC:BicepsC .

```

```

## HipC:ForearmC      .
## HipC:WristC        .
## ThighC:KneeC       .
## ThighC:AnkleC      .
## ThighC:BicepsC     .
## ThighC:ForearmC    .
## ThighC:WristC      .
## KneeC:AnkleC       .
## KneeC:BicepsC      .
## KneeC:ForearmC     .
## KneeC:WristC       .
## AnkleC:BicepsC     .
## AnkleC:ForearmC    .
## AnkleC:WristC      .
## BicepsC:ForearmC   .
## BicepsC:WristC     .
## ForearmC:WristC    .
## Over45             .
## Weight             .
## Height             .
## NeckC              .
## ChestC             .
## AbdomenC           .
## HipC               .
## ThighC             .
## KneeC              .
## AnkleC             .
## BicepsC            .
## ForearmC           .
## WristC             .
## Over45:Weight      .
## Over45:Height      .
## Over45:NeckC       .
## Over45:ChestC      .
## Over45:AbdomenC    .
## Over45:HipC        .
## Over45:ThighC      .
## Over45:KneeC       .
## Over45:AnkleC      .
## Over45:BicepsC     .
## Over45:ForearmC    .
## Over45:WristC      .
## Weight:Height      .
## Weight:NeckC       .
## Weight:ChestC      .
## Weight:AbdomenC    .
## Weight:HipC        -6.724772e-09
## Weight:ThighC      .
## Weight:KneeC       .
## Weight:AnkleC      .
## Weight:BicepsC     .
## Weight:ForearmC    .
## Weight:WristC      .
## Height:NeckC       -1.502685e-07

```



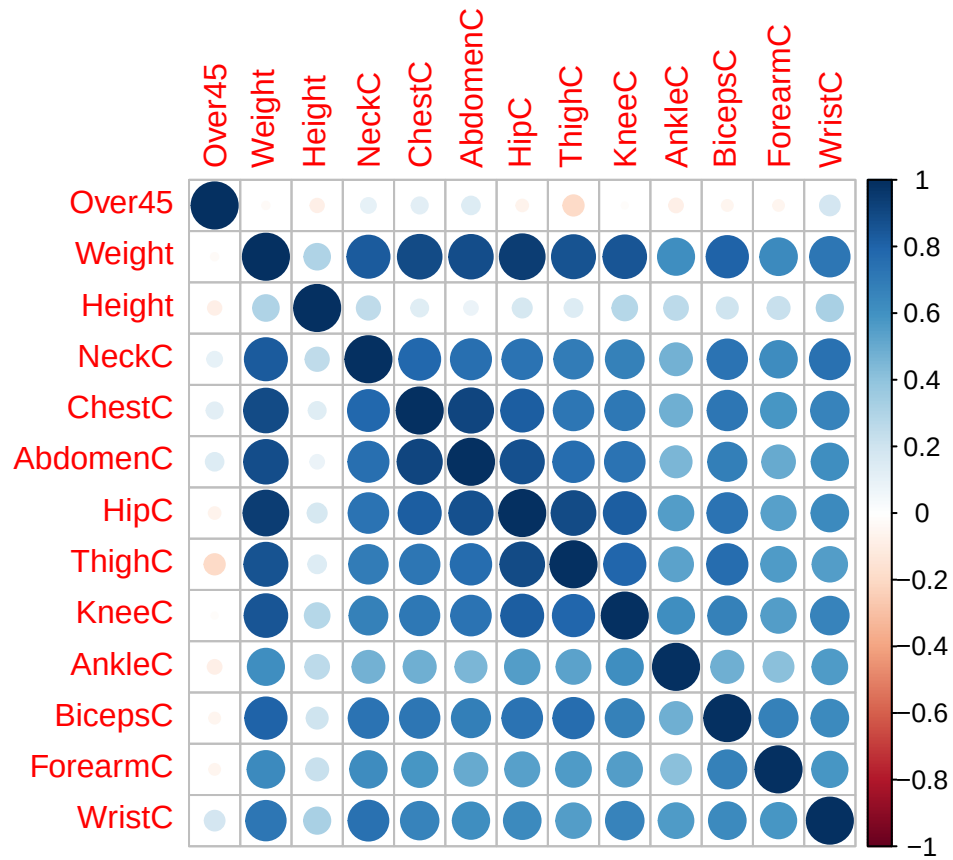
```

## Height:ChestC      .
## Height:AbdomenC    .
## Height:HipC        .
## Height:ThighC      .
## Height:KneeC       .
## Height:AnkleC      .
## Height:BicepsC     .
## Height:ForearmC    .
## Height:WristC      -4.729004e-06
## NeckC:ChestC       .
## NeckC:AbdomenC     .
## NeckC:HipC         .
## NeckC:ThighC       .
## NeckC:KneeC        .
## NeckC:AnkleC       .
## NeckC:BicepsC      .
## NeckC:ForearmC     .
## NeckC:WristC       -1.241965e-06
## ChestC:AbdomenC    .
## ChestC:HipC        .
## ChestC:ThighC      .
## ChestC:KneeC       .
## ChestC:AnkleC      .
## ChestC:BicepsC     .
## ChestC:ForearmC    .
## ChestC:WristC      .
## AbdomenC:HipC      .
## AbdomenC:ThighC    .
## AbdomenC:KneeC     .
## AbdomenC:AnkleC    .
## AbdomenC:BicepsC   .
## AbdomenC:ForearmC  .
## AbdomenC:WristC    .
## HipC:ThighC        .
## HipC:KneeC         .
## HipC:AnkleC        .
## HipC:BicepsC       .
## HipC:ForearmC      .
## HipC:WristC        .
## ThighC:KneeC       .
## ThighC:AnkleC      .
## ThighC:BicepsC     .
## ThighC:ForearmC    .
## ThighC:WristC      .
## KneeC:AnkleC       .
## KneeC:BicepsC      .
## KneeC:ForearmC     .
## KneeC:WristC       .
## AnkleC:BicepsC     .
## AnkleC:ForearmC    .
## AnkleC:WristC      .
## BicepsC:ForearmC   .
## BicepsC:WristC     .
## ForearmC:WristC    .

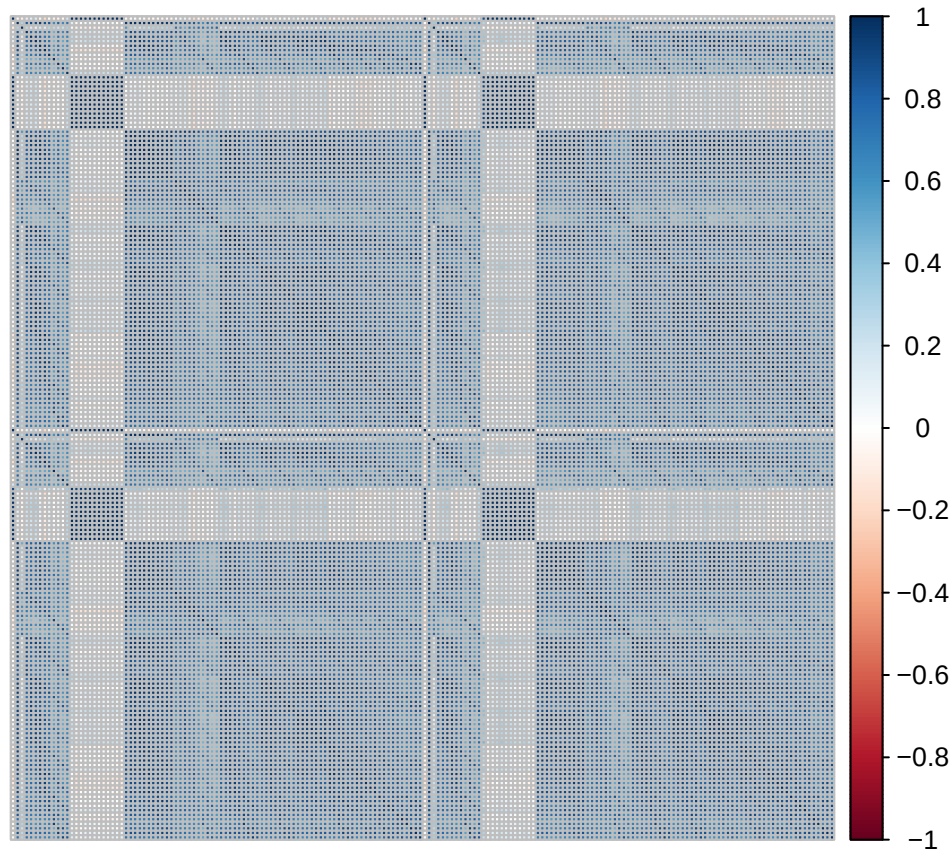
```

However, lasso assumptions are unlikely to hold due to strong correlations between predictors.

```
# original data
corrplot(cor(x_var))
```



```
# transformed data
newx = as.data.frame(newx[, 2:ncol(newx)])
corrplot(cor(newx), tl.pos='n')
```



Digging in the irrepresentability condition

As you have seen before, the variance-covariance matrix can tell us whether LASSO will work (at least asymptotically) on the data at our disposal. Let's have a look at an example (available [here](#)).

```
set.seed(12345) # Seed for replication

library(mvtnorm) # Sampling from a multivariate Normal
library(clusterGeneration) # Random matrix generation

p = 10 # = Number of Candidate Variables
k = 5 # = Number of Relevant Variables
n = 500 # = Number of observations

betas = (-1)^(1:k) # = Values for beta

sigma = genPositiveDefMat(p, "unifcorrmat")$Sigma # sigma1 violates irc
sigma1 = sigma
sigma1[9,1]=sigma1[9,1]-2
sigma1[1,9]=sigma1[1,9]-2
sigma1[10,1]=sigma1[10,1]+3
sigma1[1,10]=sigma1[1,10]+3
sigma1[7,4]=sigma1[7,4]+3
sigma1[4,7]=sigma1[4,7]+3
sigma1[6,5]=sigma1[6,5]+2
```

```

sigma1[5,6]=sigma1[5,6]+2
sigma1[10,6]=sigma1[10,6]-5
sigma1[6,10]=sigma1[6,10]-5

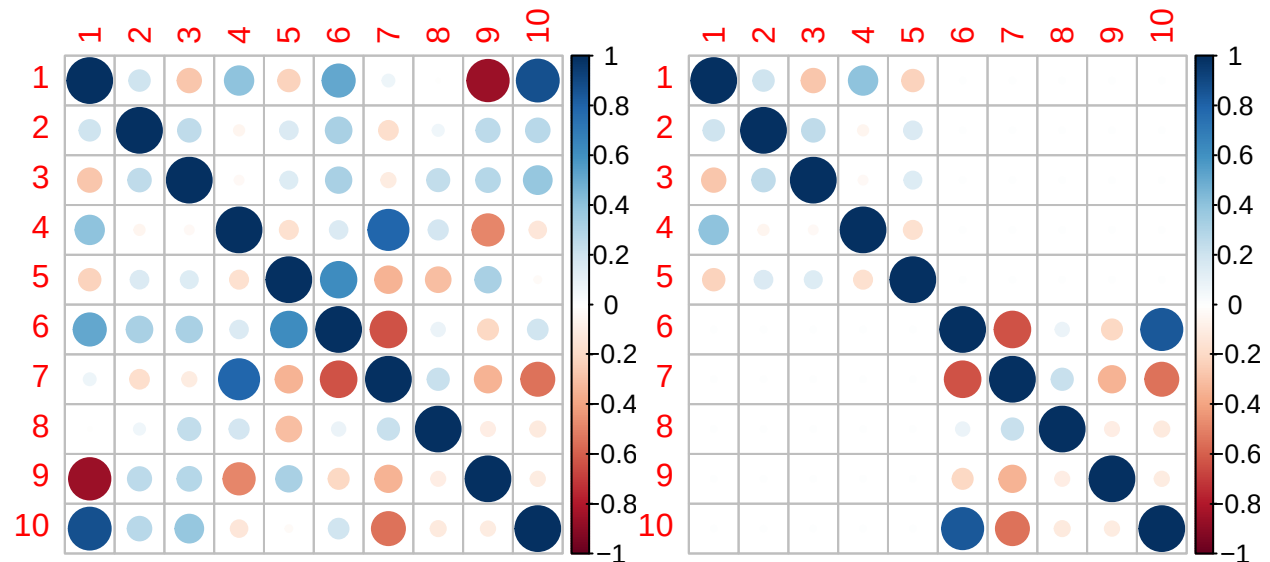
sigma2 = sigma # sigma2 satisfies irc
sigma2[(k+1):p,1:k]=0 # removing correlation among active and nonactive variables
sigma2[1:k,(k+1):p]=0

# Verify irrepresentable condition
irc1 = sort(abs(sigma1[(k+1):p,1:k] %*% solve(sigma1[1:k,1:k]) %*% sign(betas)))
irc2 = sort(abs(sigma2[(k+1):p,1:k] %*% solve(sigma2[1:k,1:k]) %*% sign(betas)))
c(max(irc1),max(irc2))

## [1] 4.886717 0.000000

# = Have a look at the correlation matrices
par(mfrow=c(1,2))
corrplot(cov2cor(sigma1))
corrplot(cov2cor(sigma2))

```



The first variance-covariance matrix does not satisfy IRC, while the second does. Let's check it “visually”, by exploring the regularization paths!

```

X1 = rmvnorm(n,sigma = sigma1) # Variables violating IRC
X2 = rmvnorm(n,sigma = sigma2) # Variables satisfying IRC
e = rnorm(n) # Error from Standard Normal
y1 = X1[,1:k]%*%betas+e # Generate y for design 1

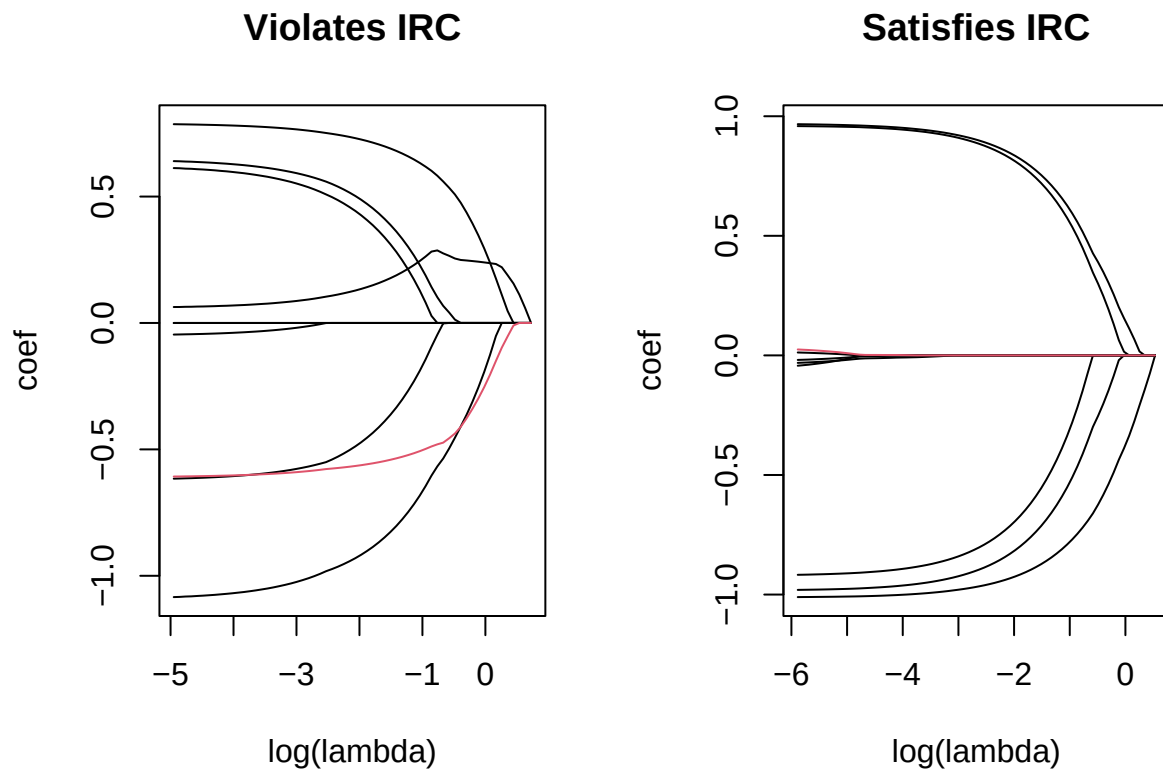
```

```

y2 = X2[,1:k]*%betas+e # Generate y for design 2
set.seed(123)
lasso1 = glmnet(X1,y1,nlambda = 100) # Estimation for design 1
cv_lasso1 = cv.glmnet(X1, y1, alpha = 1)
lasso2 = glmnet(X2,y2,nlambda = 100) # Estimation for design 2
cv_lasso2 = cv.glmnet(X2, y2, alpha = 1)

# Regularization paths
par(mfrow=c(1,2))
l1=log(lasso1$lambda)
matplot(as.matrix(l1),t(coef(lasso1)[-1,]),type="l",lty=1,col=c(rep(1,9),2),
        ylab="coef",xlab="log(lambda)",main="Violates IRC")
l2=log(lasso2$lambda)
matplot(as.matrix(l2),t(coef(lasso2)[-1,]),type="l",lty=1,col=c(rep(1,9),2),
        ylab="coef",xlab="log(lambda)",main="Satisfies IRC")

```



Nonconvex penalization methods

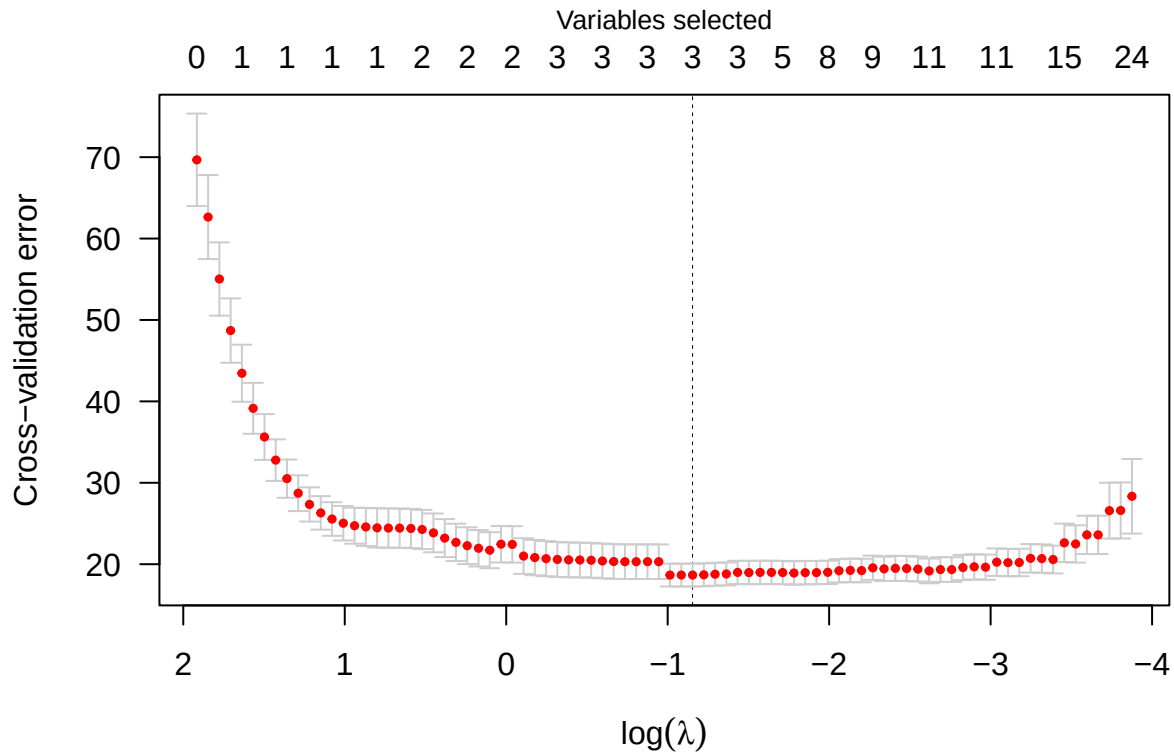
If you want to try nonconvex penalization methods, have a look at the **ncvreg** package! It has a very similar syntax than glmnet (check the paper [here](#)).

```
library(ncvreg)
```

```
dim(newx)
```

```
## [1] 252 182
```

```
cv_scad <- cv.ncvreg(as.matrix(newx), y_var)
plot(cv_scad)
```



Ridge and LASSO with Information Criterion

```
#install.packages("remotes")
#remotes::install_github("gabrielrvsc/HDeconometrics")
library(HDeconometrics)
```

We perform penalization through the **HDeconometrics** package. The function **ic.glmnet** allows us to estimate a GLM with lasso, elasticnet or ridge regularization using information criterion. It avoids some complications of cross-validation in time-series.

```
lasso_BIC1 <- ic.glmnet(X1, y1, crit = "bic")
lasso_BIC1$lambda
```

```
## [1] 0.007086288
```

```
lasso_BIC2 <- ic.glmnet(X2, y2, crit = "bic")
lasso_BIC2$lambda
```

```
## [1] 0.02589288
```

Let us rebuild the model and compare the estimated coefficients for min and 1se λ .

```
min_lasso1 <- glmnet(X1, y1, alpha=1, lambda= cv_lasso1$lambda.min)
se_lasso1 <- glmnet(X1, y1, alpha=1, lambda= cv_lasso1$lambda.1se)
```

```

min_lasso2 <- glmnet(X2, y2, alpha=1, lambda= cv_lasso2$lambda.min)
se_lasso2 <- glmnet(X2, y2, alpha=1, lambda= cv_lasso2$lambda.1se)
lasso_mat <- cbind(coef(min_lasso1), coef(se_lasso1), lasso_BIC1$coefficients,
                  coef(min_lasso2), coef(se_lasso2), lasso_BIC2$coefficients)
colnames(lasso_mat) <- c("x1_min", "x1_1se", "x1_bic",
                        "x2_min", "x2_1se", "x2_bic")
lasso_mat

```

```

## 11 x 6 sparse Matrix of class "dgCMatrix"
##           x1_min      x1_1se      x1_bic      x2_min      x2_1se
## (Intercept) -0.02097816 -0.022901129 -0.02098465 -0.0253026415 -0.02802893
## V1          .          .          .          -0.9584922227 -0.87776669
## V2          0.79087785  0.757778223  0.78616537  0.9483592572  0.88429364
## V3          -0.61320861 -0.563041198 -0.61579829 -0.8874563605 -0.77848050
## V4          0.63062063  0.530190328  0.61257868  0.9407011495  0.86856751
## V5          -1.07615734 -1.002448127 -1.08421088 -0.9986748868 -0.95657353
## V6          -0.01919053 .          .          .          .
## V7          0.04292888  0.094993145  0.06306276 -0.0080508164 .
## V8          -0.04257889 -0.009048034 -0.04589562 .          .
## V9          0.63033731  0.575012598  0.64014305 .          .
## V10         -0.61287030 -0.583711892 -0.60743277  0.0004892914 .
##           x2_bic
## (Intercept) -0.025346803
## V1          -0.952849845
## V2          0.943865492
## V3          -0.880020545
## V4          0.935815681
## V5          -0.995649309
## V6          .
## V7          -0.006753468
## V8          .
## V9          .
## V10         .

```